

Docker compose

Base problem

In the student-portal app we found that the database credentials were hardcoded in docker-compose.yml (DB_LINK and POSTGRES_PASSWORD).

```
docker run --name postgres-db \
  -e POSTGRES_USER=myuser \
  -e POSTGRES_PASSWORD=mypassword \
  -e POSTGRES_DB=mydatabase \
  -p 5432:5432 \
  -v dummy_data:/var/lib/postgresql/data \
  -d postgres:15
```

This was the main risk because secrets in plain env values are easy to expose through config files, inspection commands, and accidental sharing.

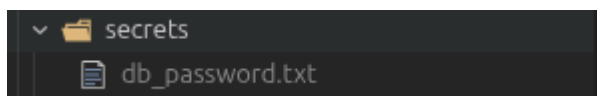
Using tmpfs mount

First, clarified the security concepts before changing code: tmpfs is still a mount, but memory-backed (RAM), not disk-backed like a normal bind/volume mount.

That means files in tmpfs are temporary and disappear when the container stops, which reduces persistence risk.

Designed a local-only flow (no external secret manager) so I can understand and test it end to end:

1. Kept a local secret file as source, then copy it at container startup into a tmpfs path used by the app and database:



2. Updated docker-compose.yml so both services (app and db) mount /run/secrets as tmpfs. This defines where runtime secrets live in memory during container execution.

```
environment:
  FLASK_APP: app.py
  FLASK_RUN_PORT: 8000
  DB_USER: postgres
  DB_HOST: db
  DB_PORT: 5432
  DB_NAME: mydb
  DB_PASSWORD_FILE: /run/secrets/db_password
  SOURCE_SECRET_PATH: /run/local-secrets/db_password.txt
  TARGET_SECRET_PATH: /run/secrets/db_password
tmpfs:
  - /run/secrets:rw,nosuid,nodev,noexec,size=1m
```

3. Added a read-only bind mount from local `./secrets` to `/run/local-secrets`. This provides the input secret file (`db_password.txt`) without making it writable inside the container and introduced a startup script (`start-with-tmpfs-secrets.sh`) that runs before each main process.

```
#!/bin/sh
set -eu

SOURCE_SECRET_PATH="${SOURCE_SECRET_PATH:-./secrets/db_password.txt}"
TARGET_SECRET_PATH="${TARGET_SECRET_PATH:-/run/secrets/db_password}"

if [ ! -f "$SOURCE_SECRET_PATH" ]; then
    echo "Secret file not found: $SOURCE_SECRET_PATH" >&2
    exit 1
fi

mkdir -p "$(dirname "$TARGET_SECRET_PATH")"
cp "$SOURCE_SECRET_PATH" "$TARGET_SECRET_PATH"
chmod 600 "$TARGET_SECRET_PATH"

exec "$@"
```

```
> Run Service
db:
  image: postgres:15
  command:
    [
      "/bin/sh",
      "/scripts/start-with-tmpfs-secrets.sh",
      "docker-entrypoint.sh",
      "postgres",
    ]
```

For PostgreSQL, replace plain `POSTGRES_PASSWORD` with `POSTGRES_PASSWORD_FILE=/run/secrets/db_password`.

For the Flask app, it stopped relying on a hardcoded full `DB_LINK` and passed DB components plus `DB_PASSWORD_FILE`. This avoids embedding the password directly in Compose while keeping connection config explicit.

```
services:
  app:
    environment:
      FLASK_APP: app.py
      FLASK_RUN_PORT: 8000
      DB_USER: postgres
      DB_HOST: db
      DB_PORT: 5432
      DB_NAME: mydb
      DB_PASSWORD_FILE: /run/secrets/db_password
      SOURCE_SECRET_PATH: /run/local-secrets/db_password.txt
      TARGET_SECRET_PATH: /run/secrets/db_password
```

4. Modified config.py to support this pattern cleanly:
it now reads password from DB_PASSWORD_FILE (or fallback env), builds the SQLAlchemy URI from parts, and still supports DB_LINK if explicitly provided. This enters in the __init__.py and allows the app to connect to the database.
5. Added .gitignore for db_password.txt so your real credential file is not committed by mistake.